Secure Handover Protocol for V2X Networks

Secure bootstrap and Secrecy Forwarding handover with Intel SGX road-side units:
architecture, implementation, validation and step-by-step guide

Repository	github.com/IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks
Clone (HTTPS)	`https://github.com/IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git`
Clone (SSH)	`git@github.com:IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git`
Group	IVNET Group 7
Date	19 October 2026

Written in C++17 with OpenSSL 3 and the Intel SGX SDK (simulation mode). Runs as separate processes on raw Ethernet sockets, inside an SGX enclave, and in an Artery + OMNeT++ + SUMO vehicular network simulation.

Contents

1. What this project is 3

2. Architecture 4

2.1 Components . . . 4

2.2 Repository layout . . . 4

2.3 Secure bootstrap . . . 5

2.4 Secrecy Forwarding (handover) . . . 6

2.5 Data path: Secure Marshaling . . . 6

3. How it was implemented 7

3.1 The enclave boundary (Enclave.edl) . . . 7

3.2 Main implementation decisions . . . 7

3.3 Artery integration (artery-sim/) . . . 8

4. Validation and results 9

4.1 What was run . . . 9

4.2 Enclave self-test (./app selftest) . . . 9

4.3 Artery simulation (D250m_80211p, 40 s) . . . 9

5. Strong points 11

6. Weak points and what can get better 12

6.1 Validation . . . 12

6.2 Design and implementation . . . 12

7. How to run it, step by step 14

7.1 What you need . . . 14

7.2 Open a terminal and learn three tricks . . . 14

7.3 Get the code . . . 14

7.4 First run: the protocol without SGX (about 5 minutes) . . . 15

7.5 Second run: the RSU inside SGX (about 30 minutes, mostly waiting) . . . 15

7.6 Third run: the Artery simulation (1-2 hours, mostly waiting) . . . 16

7.7 When something goes wrong . . . 17

1. What this project is

Cars talk to road-side units (RSUs) along the road. Before an RSU can trust a car, and before the car can trust the RSU, both need shared secret keys. This project implements a **secure bootstrap** protocol that creates those keys between a **vehicle**, an **RSU** and an **External Security Agent (ESA)**, and a **Secrecy Forwarding handover** that moves an already trusted vehicle to the next RSU on its way **without repeating the bootstrap and without asking the ESA again**.

The RSU is a **Trusted Execution Environment (TEE) gateway**: its secrets (keys, one-time passwords, the decryption of the vehicles' data) live inside an **Intel SGX enclave**, a protected area of memory that even the RSU's own operating system cannot read. After a vehicle is trusted, it sends **SmartData** (measurements such as location and sensor values); the enclave checks and decrypts it (Secure Marshaling) and forwards it, re-encrypted, to a cloud server.

The same protocol code runs in three settings:

Setting	What runs	Used for
Processes on raw sockets (repository root)	ESA, RSU and vehicle as 3 Linux processes in a Docker container; optional cloud	Checking the protocol message by message
SGX gateway (rsu-sgx-gateway/)	The RSU's secret part inside an SGX enclave; a self-test with two enclaves (two RSUs) in one process	Checking the enclave, the handover and the data path
Artery simulation (artery-sim/)	2 SGX RSUs and 1 car on a 1 km highway (OMNeT++ + SUMO), ITS-G5 radio	Checking the protocol over a realistic vehicular radio and mobility

2. Architecture

2.1 Components

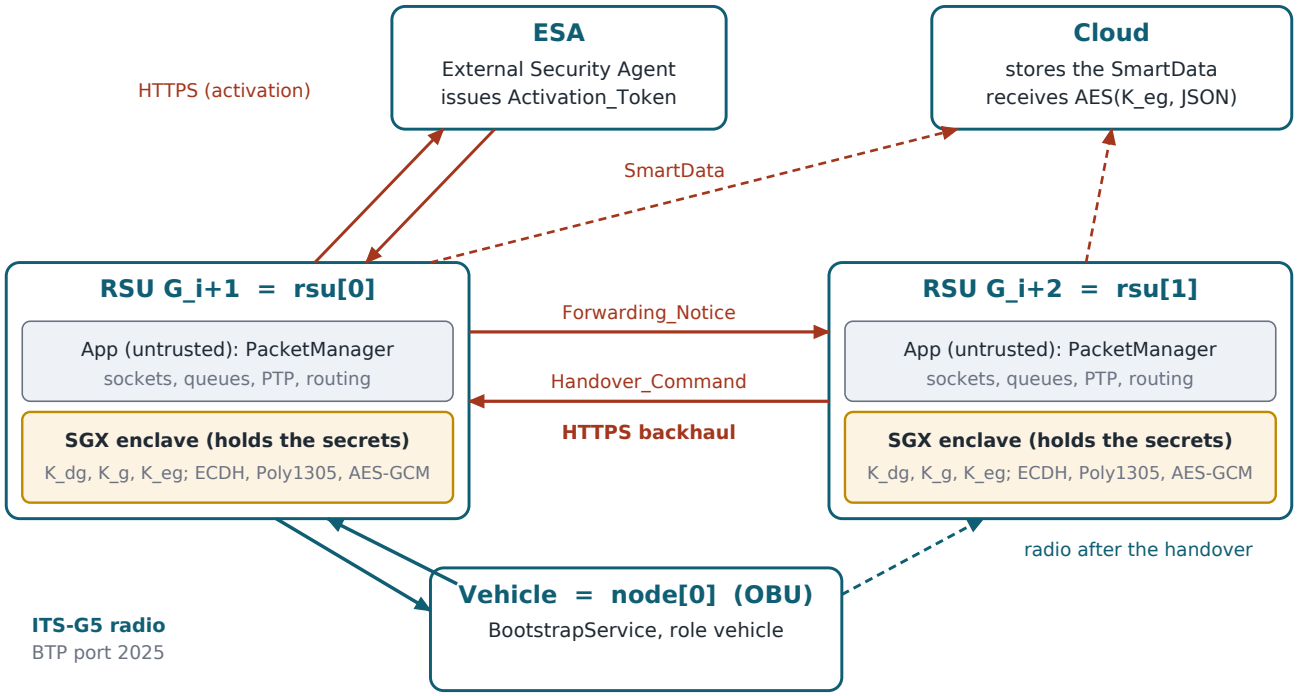


Figure 1 - Components and links. Red: HTTPS backhaul (RSU-ESA, RSU-RSU, RSU-cloud); blue: ITS-G5 radio; dashed: links used for the data path or after the handover.

Component	Role	Secrets it holds
Vehicle (OBU)	Starts the bootstrap, synchronizes its clock with the RSU (PTP), derives K_dg, sends SmartData, performs the handover	X_V (ECDH private key), K_dg, K_g, K_de, t_i
RSU gateway: App	Untrusted Linux process: sockets, OT/IT queues, PTP master, routing between radio, ESA and other RSUs	None (only what the enclave hands out on purpose)
RSU gateway: enclave	ECDH with each vehicle, OTP search, Auth_Granted, Secure Marshaling, Secrecy Forwarding, sealed state	K_dg per vehicle, K_g, K_eg, pre-computed handover keys
ESA	Authorizes activations: checks CY_g, derives Auth_d / Auth_eg, ends the protocol (Active_03)	Its key pair (CY_e), Id_d, K_de
Cloud	Receives the marshaled SmartData as AES(K_eg, JSON)	X_e (opens K_eg)

2.2 Repository layout

Path	Contents
common/	Messages (BootstrapMsg, BootstrapPayload), crypto (Crypto: SHA-256, AES-GCM, Poly1305-AES, ECDH P-256), vehicle traits, SmartData, Handover (the Secrecy Forwarding equations), SimTime, Trace
protocol/	BootstrapService (vehicle and plain RSU roles), ExternalSecurityAgent, the Bootstrap.h API
net/	Raw Ethernet sockets (AF_PACKET), framing, HTTPS client/server for the cloud

Path	Contents
main.cpp, Makefile	One process per node; make / make pretty run ESA + RSU + vehicle in Docker
rsu-sgx-gateway/App/	Untrusted side: main, PacketManager, OCALLs (per-enclave sealed state), self-test, raw-socket RSU node, benchmark
rsu-sgx-gateway/Enclave/	Trusted side: Enclave.edl (the ECALL boundary), control.cpp (bootstrap + handover), marshaling.cpp (Secure Marshaling), codec (wire formats)
artery-sim/	setup_artery.sh (installer), CMake glue, src/ (Artery modules), scenario/ (network, omnetpp.ini, SUMO highway)
setup-vm.sh	Creates the Ubuntu 24.04 VM with the SGX SDK and SGX-SSL

2.3 Secure bootstrap

A vehicle becomes **active** with an RSU through the following messages (numbers are the `BootstrapType` values). From ECDH on, every step must happen inside the activation window $[t_0, t_f]$ of 1 s.

#	Message	From → to	Content / effect
0	Activation_Request	Vehicle → RSU → ESA	CY_g (target RSU's certificate key), t_0, t_f, UUID, SHA256(f_w)
2	Activation_Token	ESA → RSU (HTTPS)	t_0, t_f, Auth_d, Auth_eg: a pending activation in the enclave
13-15	PTP Sync / Delay_Request / Delay_Response	RSU ↔ vehicle	IEEE 1588 clock synchronization (the RSU is the master)
4-5	ECDH_Request / ECDH_Response	Vehicle ↔ RSU	Y_d / Y_g: both derive K_dg (it never leaves the enclave)
6	Auth_Request	Vehicle → RSU	OTP_dt = Poly1305(K_dg, Auth_d xor t, m); the enclave tries every pending Auth_d and t ± 1 slot of 100 ms
8	Auth_Granted	RSU → vehicle	AES-256-GCM(K_dg, { OTP_dt, K_g, CY_e, Auth_eg })
10-11	Active_01 / Active_02	Vehicle → RSU → ESA	Auth_d, OTP_dt, Auth_de; both sides derive K_de
12	Active_03	ESA → user	End of the protocol

2.4 Secrecy Forwarding (handover)

Once a vehicle is active with RSU G_{i+1}, G_{i+1} announces it to the next RSU G_{i+2} right away. G_{i+2} pre-computes the handover keys in its enclave, so that when the vehicle actually arrives only a two-message exchange is needed.

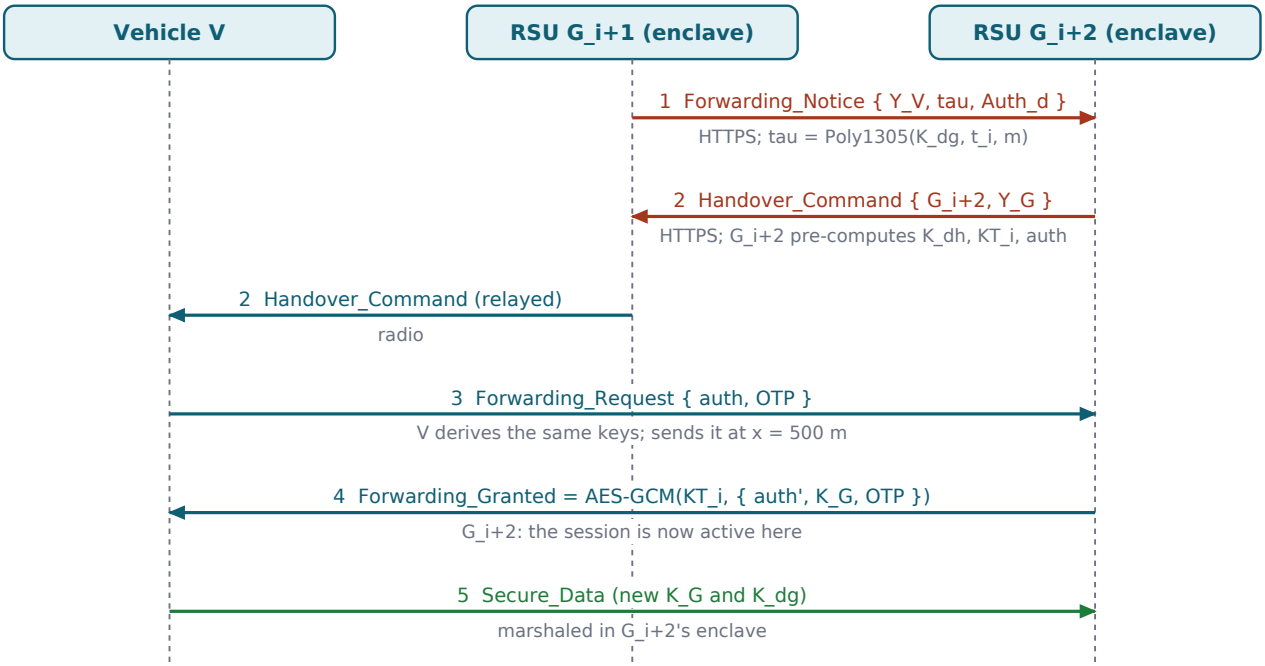


Figure 2 - Secrecy Forwarding. Steps 1-2 happen right after the bootstrap; steps 3-5 when the vehicle reaches the handover point.

The equations are in `common/Handover.h` and are compiled into both the vehicle and the enclave, so the two sides cannot drift apart:

```
tau_{i+2}   = Poly1305(K_dg, t_i, m)              t_i: slot of the OTP that activated V
K_dh_i      = ECDH(X_V, Y_G) = ECDH(X_G, Y_V)     fresh X_G per handover (G_{i+2})
KT_i        = Poly1305(K_dh_i, K_dh_i xor tau, m) 16 B: AES-128-GCM key
KT_i^32     = SHA256(KT_i || K_dh_i)              32 B: Poly1305-AES key of the OTP
auth_{i+2}  = SHA256(tau_{i+2})
OTP         = Poly1305(KT_i^32, auth xor t_c, m)  t_c: slot the request is sent in
auth'_{i+2} = SHA256(auth_{i+2})
```

After the Forwarding_Granted, K_dh_i becomes the vehicle's K_dg with G_{i+2}, the vehicle uses G_{i+2}'s group key K_G, and t_c becomes t_i for the next handover. Messages 16-20 of `BootstrapType`: Forwarding_Notice, Handover_Command, Forwarding_Request, Forwarding_Granted and Secure_Data.

2.5 Data path: Secure Marshaling

An active vehicle sends **Secure_Data**: the OT packet Message(AES(K_G, Data), Auth_d, OTP_dt), where the OTP binds the data to the vehicle's own K_dg. The enclave finds the vehicle by Auth_d, decrypts with K_G, checks the OTP (± 1 slot), builds the JSON { Location, Timing, Metadata, Value, Trust } and encrypts it with K_eg = SHA256(ECDH(X_s, Y_e)), the key of an HTTPS session with the cloud whose certificate is pinned in the enclave. A new K_eg is agreed every 100 packets. The untrusted App only carries the result and cannot read it.

3. How it was implemented

3.1 The enclave boundary (Enclave.edl)

ECALL	What the enclave does
1 ecall_init_gateway	Stores CY_g, CY_e, K_g (given or generated inside); restores the sealed state
2 ecall_control_add_activation_token	Adds a pending activation { Auth_d, t_0, t_f, Auth_eg }
3 ecall_control_handle_bootstrap	ECDH_Request → ECDH_Response; Auth_Request → Auth_Granted; Active_01 → Active_02; Forwarding_Request → Forwarding_Granted
4 ecall_secure_marshal_ot_to_it	OT packet (or Secure_Data payload) → HTTPS header | AES(K_eg, JSON)
5 ecall_trigger_handover	Source RSU: τ, Y_V and Auth_d of an active session (K_dg stays inside)
6 ecall_receive_forwarding_notice	Target RSU: fresh X_G/Y_G, pre-computes K_dh, KT_i, auth; returns Y_G
7 ecall_init_cloud_session	Pins the cloud's certificate and agrees the first K_eg

OCALLs: `ocall_print` (trace lines; the enclave never prints a secret) and `ocall_save/load_sealed_state`: the state (K_g and every active session with K_dg, Auth_d, Y_V and t_i) is sealed with `sgx_seal_data` and kept on disk by the App, which cannot read or change it unnoticed. With several enclaves in one process each one gets its own file by its enclave id (`App/ocalls.h`).

3.2 Main implementation decisions

- **One source for the equations.** `common/Handover.cpp` is linked into the vehicle, the App and the enclave, like `common/SmartData.cpp` for the data path.
- **t_i is the matched slot.** Both sides use the slot of the OTP the RSU actually matched (not the clock at a later moment), so a message that crosses a 100 ms slot boundary cannot make them disagree on τ.
- **Constant-time comparisons** (`CRYPTO_memcmp`) for every OTP, auth and Auth_d.
- **Authenticated encryption** (AES-GCM) for Auth_Granted, Active_01, Forwarding_Granted, the SmartData and the cloud payload: any change on the air fails the tag.
- **One handover per announcement.** The pre-computed handover is erased once granted, so a replayed Forwarding_Request is refused.
- **Handover_Command is relayed by G_{i+1}**: the vehicle is still in G_{i+1}'s radio range when the command is created, maybe not yet in G_{i+2}'s.
- **Option A: one enclave per RSU**, created with `sgx_create_enclave` in the same process (self-test and Artery). Each has its own memory, K_g and sealed state.
- **OMNeT++ clock bridge.** Compiled with `-DWITH_OMNETPP`, `SimTime::now()` returns the simulation time, so the protocol code is unchanged in Artery.

3.3 Artery integration (artery-sim/)

Module	Description
TeeRsuService (ItsG5Service)	One SGX enclave per RSU and the gateway's PacketManager; radio on Vanetza BTP port 2025; after Active_02 announces the vehicle to nextRsu; marshals Secure_Data and records marshaledPackets, marshalLatencyUs, handoversGranted
VehicleTeeService (ItsG5Service)	The protocol's vehicle: bootstrap with rsu[0] 0.5 s after entering, Secure_Data at 10 Hz, Forwarding_Request when x ≥ 500 m; records bootstrapDelay, handoverE2EDelayMs, secureDataSent
BackhaulBroker (cSimpleModule)	HTTPS backhaul with 20 ms latency (RSU-ESA, RSU-RSU), hosts the ESA and plays the cloud (opens every IT packet with K_eg)
HighwayHandover.ned, omnetpp.ini	artery.inet.World + backhaulBroker, rsu[0] at (250 m, 10 m), rsu[1] at (750 m, 10 m); configurations D250m_80211p (200 mW) and D500m_5G (631 mW), 5.9 GHz, 6 Mbps
SUMO files	1 km two-lane highway; node[0] enters at x = 50 m at 25 m/s, reaches x = 500 m at t ≈ 18 s and leaves at t ≈ 38 s

Radio priority: the protocol messages use traffic class TC0 (DCC profile DP0); Secure_Data uses DP2, like CAMs. ETSI DCC limits DP0 to a burst of 20 packets every 10 s, which a 10 Hz data stream would exhaust (this was found and fixed during validation, see section 4).

4. Validation and results

4.1 What was run

Test	Command	Result
Root build + 3-process run	make clean && make	0 compiler warnings; bootstrap OK (vehicle 0, rsu 0, esa 0)
Enclave self-test	make -C rsu-sgx-gateway run	0 warnings; scenarios 1-6 ok; gateway OK
Raw sockets + cloud	make -C rsu-sgx-gateway network	3 SmartData POSTs, 3 stored by the cloud
Artery, D250m_80211p	run_tee_v2x_handover	Handover done, 374/374 packets marshaled and opened
Artery, D500m_5G	run_tee_v2x_handover (RUN_FLAGS -c D500m_5G)	Not yet run at the time of writing

4.2 Enclave self-test (./app selftest)

Scenario	Expected	Observed
1. Normal bootstrap (node[0] with rsu[0])	Vehicle ready, ESA ends the protocol	ok
2. OTP changed on the air	Enclave finds no match: ignored	ok (1 refused)
3. Auth_Request after t_f	Enclave aborts the session	ok (1 refused)
4a-4g. Secure Marshaling on rsu[0]	Valid data opens to the same JSON; changed OTP, changed ciphertext, unknown Auth_d and old t refused; new K_eg after 100 packets	ok
4h-4m. Secrecy Forwarding rsu[0] → rsu[1]	Second enclave; node[0] gets rsu[1]'s K_g; its data is marshaled by rsu[1]; replayed Forwarding_Request and old keys refused	ok
5. CY_g of no gateway	ESA issues no token	ok
6. Before t_0	Enclave aborts at ECDH	ok

4.3 Artery simulation (D250m_80211p, 40 s)

Quantity	Value
Bootstrap delay (Activation_Request → vehicle ready)	43 ms (with 20 ms backhaul each way)
Handovers	1 (rsu[0] → rsu[1] at t = 18.14 s, x = 500 m)
Handover delay (Forwarding_Request → Forwarding_Granted accepted)	0.68 ms
Secure_Data sent / to rsu[0] / to rsu[1]	374 / 175 / 199
Marshaled by rsu[0] / rsu[1]; refused	175 / 199; 0
Opened by the cloud	374 of 374
Radio frames sent by the vehicle	380 of 380 produced

Bug found by the simulation. The first run used DP0 for every message. The vehicle's radio sent only 49 of 380 packets; queued packets left up to 10 s late and the enclave correctly refused 3 of them for an expired OTP; the Forwarding_Request was dropped and no handover happened. Moving Secure_Data to DP2 fixed it. This shows the value of validating over a realistic radio stack and not only in memory.

How to read the 0.68 ms. It is only the exchange at the handover point. The expensive work (ECDH, key derivation) was done in rsu[1]'s enclave at t = 0.66 s, right after the bootstrap; the 20 ms backhaul delays are therefore not in this number. ECALL processing time is wall-clock time and does not advance simulated time (it is recorded separately as marshalLatencyUs).

5. Strong points

- **Secrets stay in the enclave.** K_dg, K_g, K_eg and the handover keys are created and used only inside SGX; the App sees ciphertexts and public values. The enclave never prints a secret.
- **Fast handover by pre-computation.** The target RSU prepares K_dh, KT_i and auth before the vehicle arrives; the handover itself is two short messages (0.68 ms in the simulation), with no ESA and no new bootstrap.
- **Freshness and replay protection.** Time-slot OTPs (± 100 ms), the one-time pre-computed handover, the echoed OTP and auth' = SHA256(auth) in Forwarding_Granted.
- **Authenticated encryption everywhere** (AES-GCM) and constant-time comparisons.
- **Reboot recovery.** Sealed state per enclave restores the active sessions (with Y_V and t_i, so a vehicle can still be handed over after a restart).
- **The same code in three environments** (sockets, SGX, Artery) and one source for the handover equations, which removes a whole class of vehicle/RSU mismatches.
- **Negative tests** in the self-test: tampering, late messages, unknown identities, replays, old keys.
- **Reproducible setup.** setup-vm.sh and setup_artery.sh are idempotent and install everything from scratch; builds have no compiler warnings (-Wall -Wextra).

6. Weak points and what can get better

6.1 Validation

Weak point	How to improve it
One Artery run per configuration, one vehicle, one seed: no statistics	Repeat with several seeds (repeat = 30 in omnetpp.ini) and report means with confidence intervals
D500m_5G not run yet; it is 802.11p at 631 mW, not a real 5G NR model	Run it; for real 5G use Simu5G or label it explicitly as a power/range variant
No traffic load: a single car on an empty channel	Many vehicles, other services (CAM/DENM), several densities; measure channel busy ratio and DCC states
Handover delay excludes the pre-announcement and the backhaul	Also measure the service interruption: the gap between the last packet accepted by rsu[0] and the first by rsu[1]
No comparison baseline	Compare against a full re-bootstrap with rsu[1] (and with no TEE) for delay, messages and bytes
Fixed handover point (x = 500 m) and RSU spacing	Sweep handoverX, the RSU distance and the speed; trigger the handover on signal strength instead of a coordinate
No attacks in the simulation (only in the self-test)	Add attacker nodes: replay, forged Handover_Command, jamming, a vehicle that never arrives
SGX simulation mode only	Measure ECALL latency on SGX hardware (SGX_MODE=HW) and add remote attestation
Scenario tests, no unit tests or CI	Unit tests for Crypto, Handover and the codecs; fuzz the parsers; GitHub Actions running make and the self-test (SIM mode)
Security argued informally	Model the handover in ProVerif or Tamarin (secrecy of K_dh, authentication, replay)

6.2 Design and implementation

- **τ leaves the source enclave.** It is handed to the untrusted App, which sends it over the backhaul. It cannot forge the handover alone (an ECDH private key is still needed), but it should travel encrypted enclave-to-enclave.
- **Handover_Command is not authenticated.** A forged one cannot complete a handover, but it can make the vehicle waste a request; it could be signed by the source enclave.
- **The source RSU keeps the session** after the handover (no revocation message), and the optional pre-load of the next RSU's key (Y_G_i+3) is not implemented.
- **The vehicle prints its keys** in its trace (demonstration output) and sends the Activation_Request in place of the operator, so UUID and SHA256(f_w) cross the radio in clear.
- **PTP timestamps** are taken when a message is handed to the stack, not when the radio transmits it; network queues (DCC) add error.
- **The plain RSU** (BootstrapService in the RSU role) has no handover; only the SGX gateway does.
- **One OTP window** (± 1 slot of 100 ms) for everything: radio queuing delays can push valid packets outside it.

- **The Artery build** was verified on the Ubuntu 24.04 VM only; the Arch Linux and opp_env paths of setup_artery.sh were not tested.

7. How to run it, step by step

> This section assumes no previous experience. Follow the steps in order and copy the commands exactly. Everything is typed in a **terminal**: a window where you type commands and press **Enter** to run them.

7.1 What you need

- A computer with **Ubuntu 24.04**, connected to the internet. (The commands below are for Ubuntu; setup-vm.sh itself also supports Fedora and Arch Linux.)
- **8 GB of memory** or more, and about **50 GB of free disk space**.
- A processor with **virtualization enabled** (Intel VT-x or AMD-V; it is usually on; if step 7.5 says KVM is missing, enable it in the computer's BIOS/UEFI settings).
- Your user password (the computer asks for it when installing software).
- Time: about 30 minutes for parts 7.2-7.4, 1-2 hours more for the simulation (7.6), mostly waiting.

7.2 Open a terminal and learn three tricks

1. Press **Ctrl + Alt + T** together (or open the application list and search for **Terminal**). A window with a blinking cursor opens.
2. To **paste** a command copied from this document, click in the terminal and press **Ctrl + Shift + V** (in a terminal Ctrl + V alone does not paste). Then press **Enter**.
3. When asked for a **password**, type it and press Enter: **nothing appears while you type**, that is normal. When a command asks Do you want to continue? [Y/n], type **Y** and press Enter.
4. Wait until the line ending in **$** appears again before typing the next command. Lines starting with **#** in this document are comments: you do not need to type them.

7.3 Get the code

1. Install git (the tool that downloads the code) and the build tools:

```
sudo apt update
sudo apt install -y git build-essential libssl-dev docker.io
```

2. Download the project. Use the **HTTPS** link (works for everyone):

```
cd ~
git clone https://github.com/IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git
```

If you have an SSH key registered on your GitHub account you can use the SSH link instead:

```
git clone git@github.com:IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git
```

3. Enter the project folder (every later command of 7.4 and 7.5 starts from here):

```
cd ~/Secure-Handover-Protocol-For-V2X-Networks
```

7.4 First run: the protocol without SGX (about 5 minutes)

1. Allow your user to use Docker, then **log out of the computer and log in again** (or restart it):

```
sudo usermod -aG docker $USER
```

2. Open a terminal again, enter the folder and run the three nodes (vehicle, RSU, ESA):

```
cd ~/Secure-Handover-Protocol-For-V2X-Networks
make pretty
```

> **Success looks like:** many lines describing each protocol step, ending with `bootstrap OK (exit codes: vehicle 0, rsu 0, esa 0)`.

7.5 Second run: the RSU inside SGX (about 30 minutes, mostly waiting)

SGX needs a special software kit, which is installed inside a **virtual machine** (VM): a computer simulated inside your computer, so nothing on your own system has to change.

1. Create an SSH key (it lets your computer enter the VM without a password). Press **Enter** three times when it asks questions. If it says the file already exists, type **n** and continue.

```
ssh-keygen -t ed25519
```

2. Create the VM. This installs the virtualization software, downloads Ubuntu and installs the SGX kit inside the VM. It takes 10-25 minutes:

```
./setup-vm.sh
```

> **Success looks like:** `=== Success! Environment configured and VM 'ubuntu24-tee-vm' is running ===`. Then **log out and log in again** once (so your user may control VMs).

3. Build and test the SGX gateway. The command copies the project into the VM (to the folder ~/v2x there), builds it and runs the self-test:

```
cd ~/Secure-Handover-Protocol-For-V2X-Networks/rsu-sgx-gateway
make run
```

> **Success looks like:** a results table with six lines ending in **ok** (including `4. handover rsu[0] -> rsu[1] ... Secrecy Forwarding (4h-4m): ok`) and the last line `gateway OK`.

4. Optional: the same RSU on the network with a cloud server (it may ask for your password once):

```
make network
```

7.6 Third run: the Artery simulation (1-2 hours, mostly waiting)

The simulation is built and run **inside the VM**, because that is where the SGX kit is.

1. Make sure the VM has the latest copy of the project (step 7.5.3 does it; run it again whenever you change the code on your computer):

```
make -C ~/Secure-Handover-Protocol-For-V2X-Networks/rsu-sgx-gateway
```

2. Find the VM's network address and keep it in the variable `VM` (the second line prints it, e.g. 192.168.122.69), then enter the VM. The prompt changes to `ubuntu@ubuntu24-tee-vm`: from now on the commands run inside the VM. Type **exit** to come back to your computer. If it prints nothing or says domain is not running, start the VM first with `virsh -c qemu:///system start ubuntu24-tee-vm`, wait one minute and repeat this step.

```
VM=$(virsh -c qemu:///system domifaddr ubuntu24-tee-vm \
     | awk '/ipv4/ {split($4, a, "/"); print a[1]}')
echo $VM
ssh ubuntu@$VM
```

3. Install OMNeT++, Artery and SUMO and build the scenario. **Do not close the window** while it runs; if it stops (for example the connection drops), run the same command again: it continues where it stopped.

```
cd ~/v2x/artery-sim
./setup_artery.sh --install-sgx
```

Success looks like: `===  done  ===` followed by the two commands to run the simulation.

4. Run the simulation (first configuration, 802.11p at 200 mW):

```
cd ~/v2x/artery-sim/deps/artery/build
cmake -DRUN_FLAGS="-u Cmdenv -c D250m_80211p" .
cmake --build . --target run_tee_v2x_handover
```

Success looks like (about one minute): `node[0] -> vehicle ready (with rsu[0])` near t = 0.64 s, `rsu[1] -> handover done (node[0] now with rsu[1])` near t = 18.14 s, and finally `Simulation time limit reached -- at t=40s`.

5. See the numbers of section 4.3:

```
cd ~/v2x/artery-sim/scenario/results
grep -E "TeeRsuService|VehicleTeeService|backhaulBroker" D250m_80211p-0.sca
```

6. Run the second configuration (631 mW) the same way:

```
cd ~/v2x/artery-sim/deps/artery/build
cmake -DRUN_FLAGS="-u Cmdenv -c D500m_5G" .
cmake --build . --target run_tee_v2x_handover
```

7. Copy the results to your own computer: type **exit** to leave the VM, then (in the same terminal, so that `VM` still holds the address):

```
mkdir -p ~/v2x-results
scp "ubuntu@$VM:v2x/artery-sim/scenario/results/*" ~/v2x-results/
```

7.7 When something goes wrong

Message	What to do
permission denied ... docker.sock	You skipped the log out / log in after usermod (7.4.1).
network 'default' is not active	sudo virsh net-start default && sudo virsh net-autostart default
Host key verification failed (ssh)	ssh-keygen -R $VM (the VM's address, step 7.6.2), then try again and answer yes
configure: error: Before running 'configure', please use 'source setenv'	The VM has an old copy of the project: run step 7.6.1 on your computer, then 7.6.3 again.
Could NOT find Python (missing: Python_INCLUDE_DIRS ...)	Old copy of the project (same fix), or install it by hand in the VM: sudo apt install -y python3-dev
Cannot load library ... undefined symbol	A build was interrupted and left empty files. In the VM: cd ~/v2x/artery-sim/deps/artery/build, then find . -name '*.o' -size 0 -delete, then cmake --build . -j 2
tee_v2x_handover skipped: no Intel SGX SDK	Run setup_artery.sh with --install-sgx (inside the VM).
The VM does not answer (ssh timeout)	virsh -c qemu:///system start ubuntu24-tee-vm, wait one minute, try again

Do not run two builds of the simulation at the same time (for example two terminals running setup_artery.sh): they overwrite each other's files.